

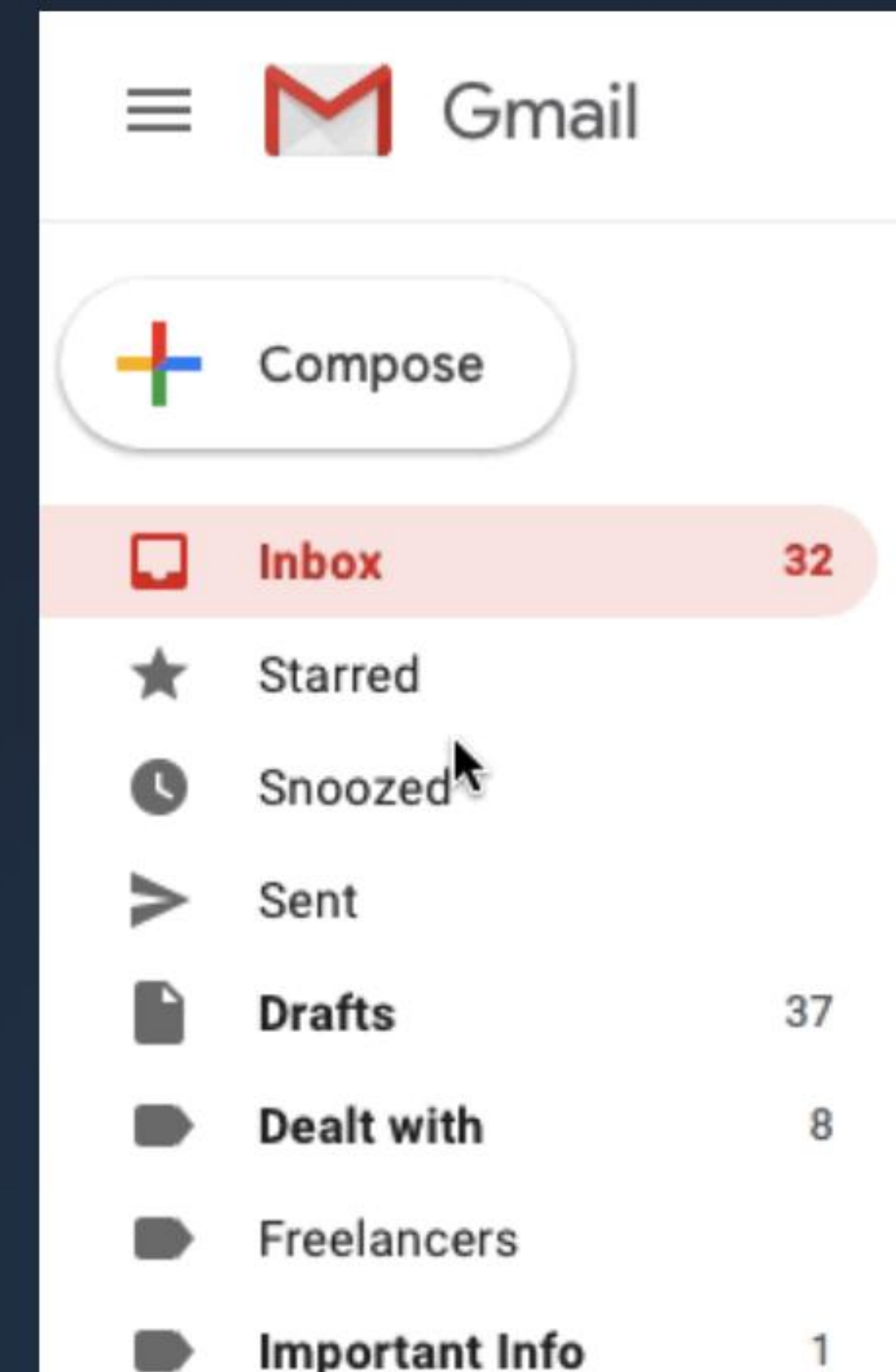
Naive Bayes: The Ultimate Spam Filter

Building a high-accuracy text classifier from your notebook

The Problem: Ham vs. Spam

Our dataset contains thousands of messages, each labeled as 'ham' (legitimate) or 'spam' (unwanted).

- **Ham:** "Ok lar... Joking wif u oni..."
- **Spam:** "Free entry in 2 a wkly comp..."
- **Goal:** Build a model that can automatically classify any *new* message.



The Big Challenge: How Do Computers Read Text?

Models don't understand words. They only understand numbers.

Step 1: The "Bag of Words"

The Message

```
'Free entry in 2 a wkly comp to win...'
```

The Process: CountVectorizer

This tool from your notebook converts text into a "vector" by counting word occurrences.

The "Vector" (The Numbers)

It creates a giant list of all words, and counts how many times each appears in one message.

```
[... free: 1, entry: 1, wkly: 1, comp: 1, win: 1  
...]
```

These counts become the features for our model.

Step 2: The Model (Multinomial Naive Bayes)



Why "Multinomial"? Because it's designed to work with **counts** (e.g., word counts). This is perfect for our "Bag of Words".



How it "Thinks": It calculates the probability of a message being spam **given** the words it contains.



The "Naive" Part: It assumes words like "free" and "win" are independent. This is wrong, but it makes the model **insanely** fast.

How It "Thinks": The Probability Game

“

'Free', 'discount', 'reward', 'win'...

These words have a high probability of appearing in spam.

'Ok', 'home', 'mohan', 'tomorrow'...

have a high probability of appearing in ham.

”

— Naive Bayes just multiplies these probabilities to make a final guess.

Why Naive Bayes is KING for Text



Insanely Fast

It trains in seconds, even on millions of emails. Perfect for real-time filtering.



Handles High Dimensions

Our "Bag of Words" can have 20,000+ features (words). NB handles this easily, while models like KNN would fail.



Incredibly Effective

Despite its "naive" assumption, it's a top-performing algorithm for text classification.

The "Easy Button": Your Sklearn Pipeline

Step 1: `CountVectorizer()`

This part of the pipeline automatically turns raw text messages into number vectors.

```
'Upto 20% discount...'
```

```
[... discount: 1, upto: 1, 20: 1 ...]
```

Step 2: `MultinomialNB()`

This part of the pipeline takes the number vector and feeds it to the Naive Bayes model to get a prediction.

```
[... discount: 1, upto: 1, 20: 1 ...]
```

```
PREDICTION: SPAM (1)
```


The Result (From Your Notebook)

98.2%

Model Accuracy

Outstanding Performance

This score shows that our model, using the `Pipeline`, correctly classified over 98% of the test messages. This is an excellent result for such a fast and simple model, proving its power for this task.

Real-World Test (From Your Notebook)

Prediction: HAM [0]

```
'Hey mohan, can we get together to watch football  
game tomorrow?'
```

Correct! The model identified this as a legitimate message.

Prediction: SPAM [1]

```
'Upto 20% discount on parking, exclusive offer  
just for you. Dont miss this reward!'
```

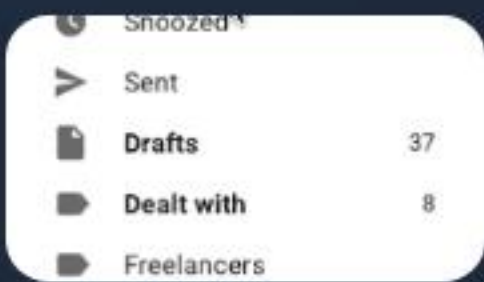
Correct! The model identified words like "discount", "exclusive", and "reward" as strong spam indicators.

Conclusion: Why We Chose Naive Bayes

- ✓ **It's the right tool for the job.** It's specifically designed for text-based probability (MultinomialNB).
- ✓ **It beats complexity.** It doesn't get confused by thousands of features (words) like other models might.
- ✓ **It's efficient.** The `Pipeline` from your notebook shows how it can be built into a clean, fast, and highly accurate system.

Questions?

Image Sources



<https://public-assets.typeform.com/helpcenter/uploads/2019/01/21143834/spam-11.gif>

Source: help.typeform.com